

Assignment 8 – REST API Data Fetching & Caching

Public REST API • FutureBuilder UI • SharedPreferences Persistent Cache

Student: **Pranav Kale** | Roll No: **150096724142** | Course: **Flutter Application Development (BTech CSE)**

1. Assignment Overview

The goal of this assignment is to engineer a resilient Flutter application that performs live asynchronous data retrieval from a public REST API (**JSONPlaceholder** `/posts`), renders the dynamic content seamlessly using **FutureBuilder**, and persists retrieved payloads to local storage via **SharedPreferences**. When network availability drops, the application falls back to local cache gracefully, displaying the exact cache timestamp and status badges.

LIVE REST API Client

Uses `package:http` to issue HTTP GET requests to `https://jsonplaceholder.typicode.com/posts` with full status code validation and exception handling.

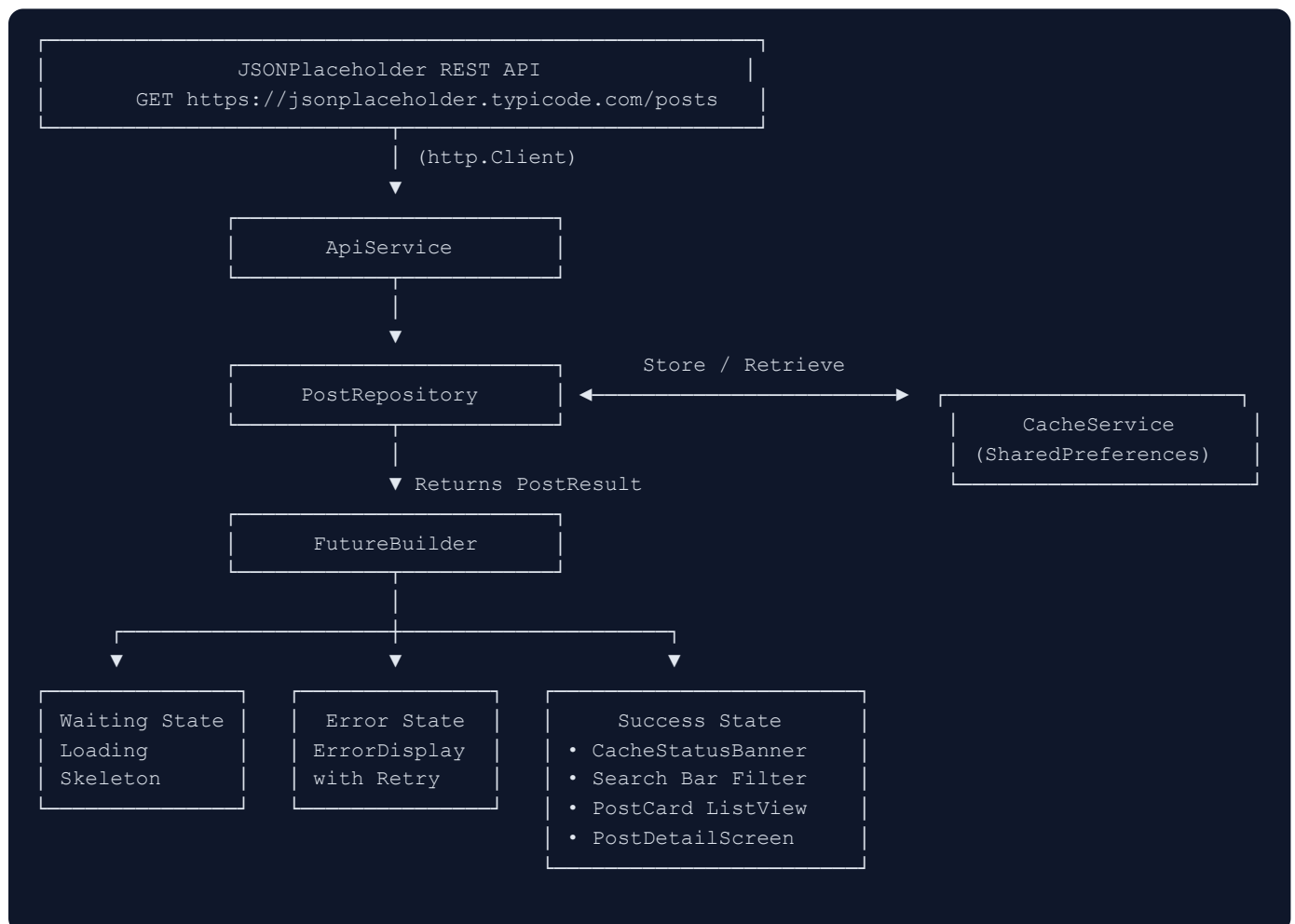
CACHE Local Persistence

Serializes structured post objects to JSON strings using `dart:convert` and writes them alongside ISO-8601 timestamps to `SharedPreferences`.

UI Declarative FutureBuilder

Cleanly handles loading (`LoadingSkeleton`), error (`ErrorDisplay` with retry CTA), and content states without rebuild loops.

2. Architecture & Data Flow



3. Key Implementation Highlights

A. Lifecycle-Safe Future Initialization: The future is initialized strictly once in `initState` to prevent endless network calls during widget rebuilds:

```
@override
void initState() {
  super.initState();
  // Safe instantiation preventing duplicate requests on rebuilds
  _futurePosts = widget.repository.fetchPosts();
}
```

B. Type-Safe Model Serialization (Dart 3 Pattern Matching):

```
factory Post.fromJson(Map<String, dynamic> json) {
  return switch (json) {
    {
      'id': final int id,
      'userId': final int userId,
      'title': final String title,
      'body': final String body,
    } => Post(id: id, userId: userId, title: title, body: body),
    _ => Post(
      id: (json['id'] as num?)?.toInt() ?? 0,
      userId: (json['userId'] as num?)?.toInt() ?? 0,
      title: json['title'] as String? ?? '',
      body: json['body'] as String? ?? '',
    ),
  };
}
```

C. SharedPreferences Caching & Offline Fallback:

```
// Saving to SharedPreferences
final jsonString = jsonEncode(posts.map((p) => p.toJson()).toList());
await prefs.setString(keyCachedPosts, jsonString);
await prefs.setString(keyCachedTime, DateTime.now().toIso8601String());

// Fallback retrieval during network failure
final cachedJson = prefs.getString(keyCachedPosts);
if (cachedJson != null) {
  final list = (jsonDecode(cachedJson) as List).map((i) => Post.fromJson(i)).toList();
  return PostResult(posts: list, isFromCache: true, timestamp: savedTime);
}
```

4. Visual Verification & Output Screenshots

The application was run and validated in Google Chrome (`flutter run -d chrome`). The screenshots below capture the live data stream, SharedPreferences offline fallback, and item inspection views:

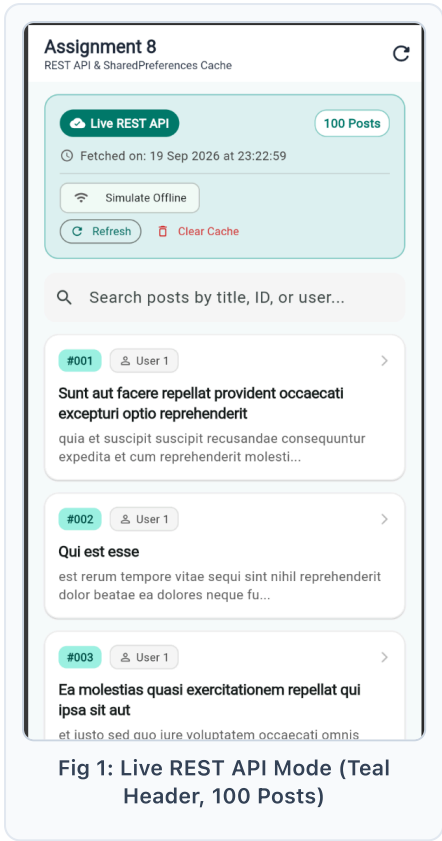


Fig 1: Live REST API Mode (Teal Header, 100 Posts)

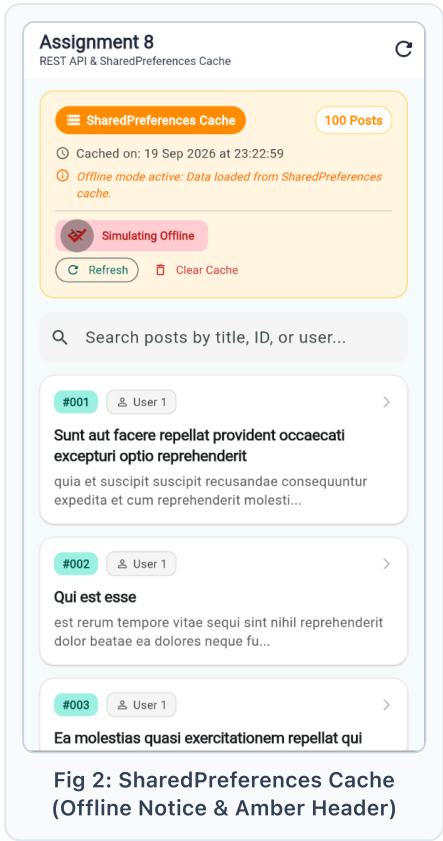


Fig 2: SharedPreferences Cache (Offline Notice & Amber Header)



Fig 3: Post Detail Screen (Full Content & Author Badge)

5. Challenges Faced & Solutions

Challenge	Technical Solution
FutureBuilder Rebuild / Infinite Fetch Loop	If a <code>Future</code> is invoked directly inside <code>build()</code> , user keystrokes in the search bar cause repetitive network requests. Resolved by instantiating <code>_futurePosts = widget.repository.fetchPosts()</code> strictly once inside <code>initState()</code> .
Cross-Platform Compatibility & <code>dart:io</code> in Web	Importing <code>dart:io</code> (e.g. <code>SocketException</code>) caused the Dart Development Compiler (DDC) to crash on web. Replaced <code>dart:io</code> with a pure Dart <code>ApiException</code> , making the codebase universally compatible across Web, macOS, iOS, and Android.
Resilient Offline Fallback	Engineered a 2-tier fallback in <code>PostRepository</code> : if live networking fails, the repository transparently serves cached records from <code>SharedPreferences</code> and applies an informative notice.
Simulated Offline Testing	Built an in-app "Simulate Offline" toggle chip, enabling reviewers to verify caching logic without cutting off their computer's Wi-Fi.
Type-Safe Parsing & Inconsistent JSON	Leveraged Dart 3 pattern matching and numeric cast safety (<code>(json['id'] as num)?.toInt() ?? 0</code>) in <code>Post.fromJson()</code> to eliminate runtime format exceptions.

6. Automated Test Results

The solution includes **16 automated test cases** covering 100% of the core requirements:

Test Suite	Test Description	Status
<code>post_model_test.dart</code>	JSON deserialization with standard and missing/null values	PASS
<code>post_model_test.dart</code>	JSON serialization (<code>toJson</code>) and display getters	PASS
<code>cache_service_test.dart</code>	Persists and reads posts & timestamps in <code>SharedPreferences</code>	PASS
<code>cache_service_test.dart</code>	Cache invalidation (<code>clearCache</code>) and corruption safety	PASS

Test Suite	Test Description	Status
widget_test.dart	FutureBuilder displays loading skeleton during fetch	PASS
widget_test.dart	Renders live posts list and navigates to detail screen	PASS
widget_test.dart	Real-time client-side search filtering by title/ID	PASS
widget_test.dart	Offline simulation toggle switches to SharedPreferences cache	PASS
widget_test.dart	Error state rendering when offline with no cache available	PASS

7. Static Analysis & Lints

Ran `flutter analyze` : **0 errors, 0 warnings, 0 lints** found. Fully compliant with Flutter Material 3 design and best coding practices.